

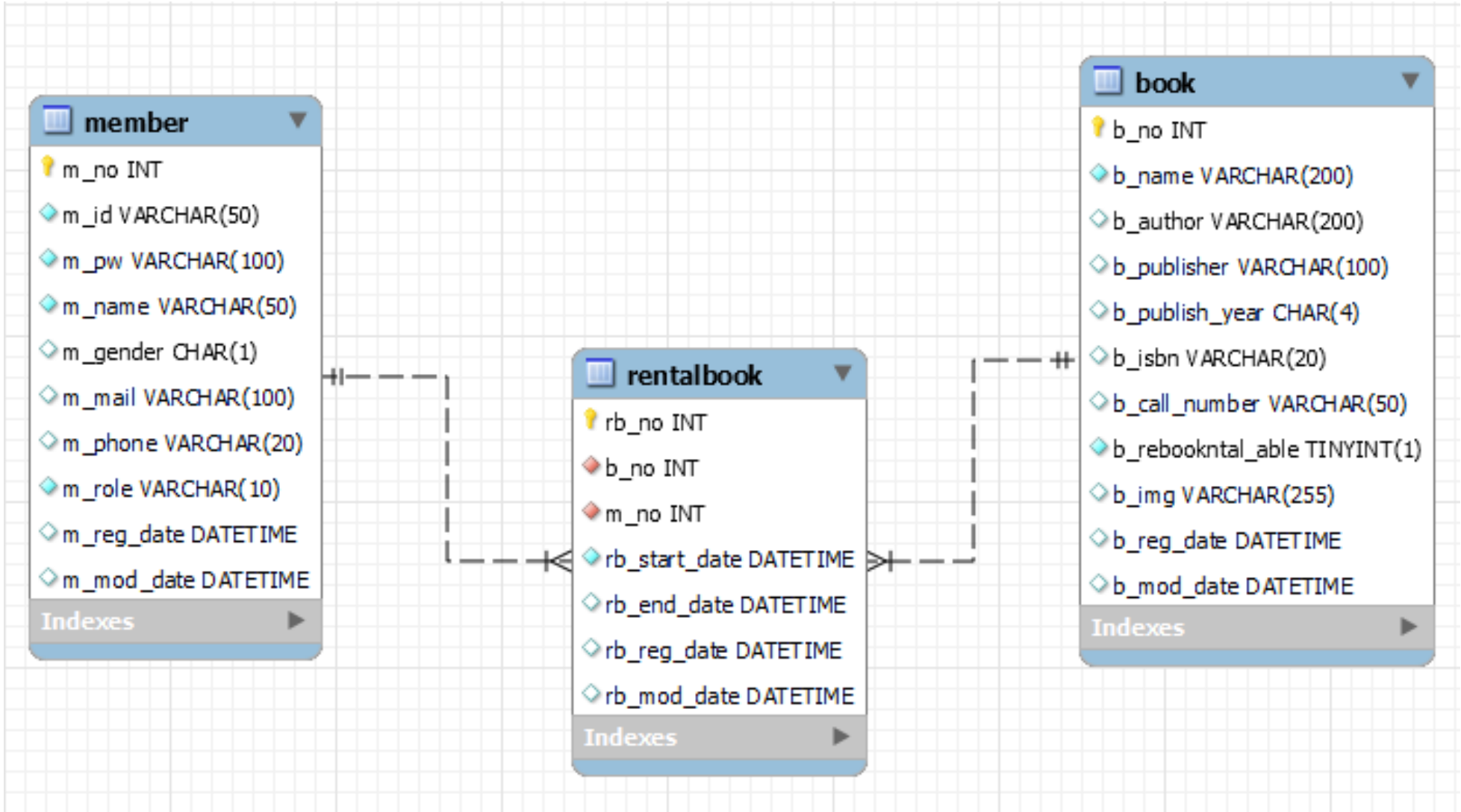
Spring Framework



데이터베이스 구축

데이터베이스 구축

- ERD



데이터베이스 구축

- aitech_library DataBase 생성

```
1· CREATE DATABASE IF NOT EXISTS aitech_library
2    DEFAULT CHARACTER SET utf8mb4
3    DEFAULT COLLATE utf8mb4_general_ci;
4
5· USE aitech_library;
```

데이터베이스 구축

- member Table Schemas

```sql
CREATE TABLE member (
    m_no        INT AUTO_INCREMENT PRIMARY KEY,
    m_id        VARCHAR(50)  NOT NULL UNIQUE,
    m_pw        VARCHAR(100) NOT NULL,
    m_name      VARCHAR(50)  NOT NULL,
    m_gender    CHAR(1),
    m_mail      VARCHAR(100),
    m_phone     VARCHAR(20),
    m_role      VARCHAR(10)  NOT NULL DEFAULT 'USER',
    m_reg_date DATETIME,
    m_mod_date DATETIME
);
```

데이터베이스 구축

- book Table Schemas

```sql
CREATE TABLE book (
    b_no            INT AUTO_INCREMENT PRIMARY KEY,
    b_name          VARCHAR(200) NOT NULL,
    b_author        VARCHAR(200),
    b_publisher     VARCHAR(100),
    b_publish_year  CHAR(4),
    b_isbn          VARCHAR(20),
    b_call_number   VARCHAR(50),
    b_rental_able   TINYINT(1) NOT NULL DEFAULT 1,
    b_img           VARCHAR(255),
    b_reg_date      DATETIME,
    b_mod_date      DATETIME
);
```

데이터베이스 구축

- book Table Schemas

```sql
CREATE TABLE rentalbook (
    rb_no          INT AUTO_INCREMENT PRIMARY KEY,
    b_no           INT NOT NULL,
    m_no           INT NOT NULL,
    rb_start_date DATETIME NOT NULL,
    rb_end_date    DATETIME NULL,
    rb_reg_date    DATETIME,
    rb_mod_date    DATETIME,
    FOREIGN KEY (b_no) REFERENCES book(b_no),
    FOREIGN KEY (m_no) REFERENCES member(m_no)
);
```

데이터베이스 연결

데이터베이스 연결

- pom.xml 의존성 추가
  - https://mvnrepository.com/에서 검색
    - ✓ spring jdbc - Spring Boot Starter JDBC / mysql - MySQL Connector/J
    - ✓ Jakarta jstl - Jakarta Standard Tag Library Implementation
      Jakarta Standard Tag Library API
    - ✓ scope 생략하면 기본값 compile은 실행 중, 로딩 시는 runtime / vesion 생략 시 parent를 따름

데이터베이스 연결

- pom.xml 의존성 추가

```
60          <!-- Source: https://mvnrepository.com/artifact/org.springframework.boot/spring-boot-starter-jdbc -->
61          <dependency>
62              <groupId>org.springframework.boot</groupId>
63              <artifactId>spring-boot-starter-jdbc</artifactId>
64          </dependency>
65          <!-- Source: https://mvnrepository.com/artifact/com.mysql/mysql-connector-j -->
66          <dependency>
67              <groupId>com.mysql</groupId>
68              <artifactId>mysql-connector-j</artifactId>
69              <scope>runtime</scope>
70          </dependency>
71          <!-- Source: https://mvnrepository.com/artifact/org.glassfish.web/jakarta.servlet.jsp.jstl -->
72          <dependency>
73              <groupId>org.glassfish.web</groupId>
74              <artifactId>jakarta.servlet.jsp.jstl</artifactId>
75          </dependency>
76          <!-- Source: https://mvnrepository.com/artifact/jakarta.servlet.jsp.jstl/jakarta.servlet.jsp.jstl-api -->
77          <dependency>
78              <groupId>jakarta.servlet.jsp.jstl</groupId>
79              <artifactId>jakarta.servlet.jsp.jstl-api</artifactId>
80          </dependency>
```

데이터베이스 연결

- DB연결 정보 생성(db.properties)

```
db.driver=com.mysql.cj.jdbc.Driver
db.url=jdbc:mysql://localhost:3306/aitech_library26?serverTimezone=UTC&characterEncoding=UTF-8
db.username=root
db.password=1234
```

- DBConfig.java 생성
  - @Configuration 설정 정보와 관련한 어노테이션
  - @Bean 메서드가 리턴한 객체를 빈으로 등록하는 어노테이션
  - @PropertySource 외부 설정 값 불러오는 어노테이션
  - @Value 파일의 값을 ${키}로 찾아 변수에 넣는 어노테이션
  - @Controller 웹 요청(URL)을 받아 처리하는 어노테이션
  - @Repository DB 접근 담당 클래스 어노테이션
  - @Autowired 등록된 Bean을 찾아 자동으로 주입 어노테이션

데이터베이스 연결

```
11 @Configuration
12 @PropertySource("classpath:db.properties")
13 public class DBConfig {
14     @Value("${db.driver}")
15     private String driver;
16
17     @Value("${db.url}")
18     private String url;
19
20     @Value("${db.username}")
21     private String username;
22
23     @Value("${db.password}")
24     private String password;
25
26     @Bean
27     public DataSource dataSource() {
28         DriverManagerDataSource dataSource = new DriverManagerDataSource();
29         dataSource.setDriverClassName(driver);
30         dataSource.setUrl(url);
31         dataSource.setUsername(username);
32         dataSource.setPassword(password);
33         return dataSource;
34     }
35 }
```

데이터베이스 연결

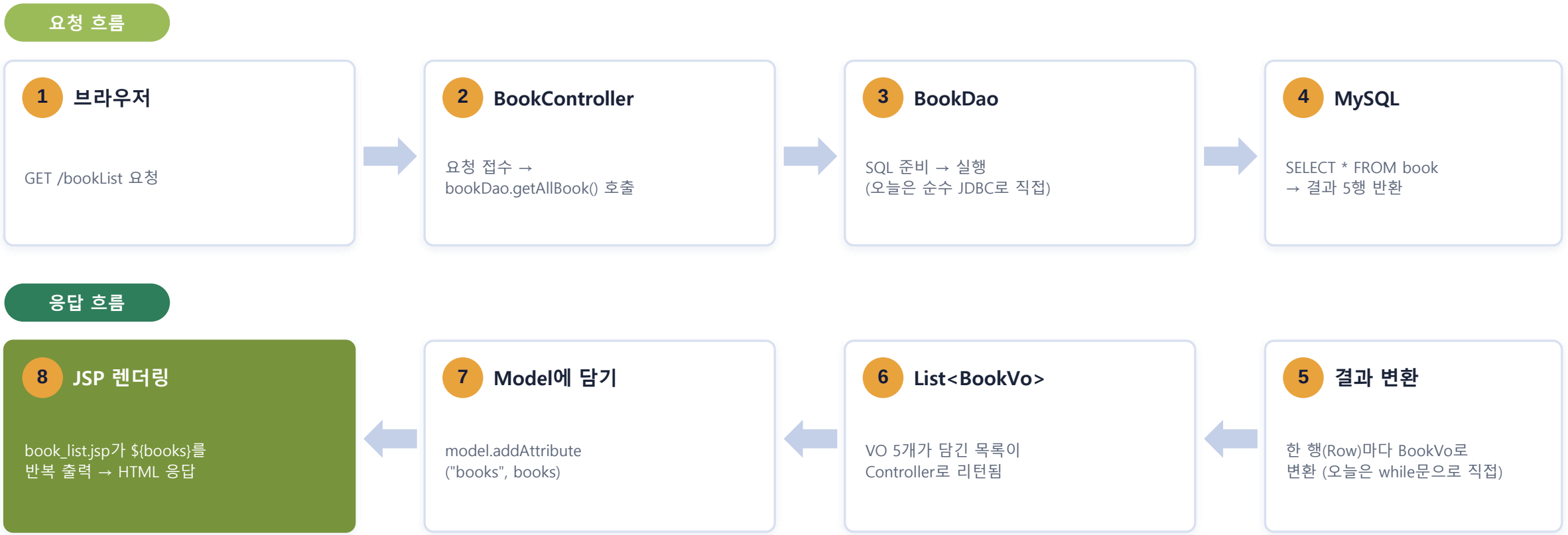
- DB연결 확인 DBTestController

```
13@Controller
14 public class DBTestController {
15
16@    @Autowired
17     private DataSource dataSource;
18
19@    @RequestMapping("/dbtest")
20     @ResponseBody
21     public String dbtest() {
22         Connection conn;
23@        try {
24             conn = dataSource.getConnection();
25             String schema = conn.getCatalog();
26             conn.close();
27             return "DB 연결 성공 접속 스키마 = " + schema;
28@        } catch (SQLException e) {
29             return "연결 실패 " + e.getMessage();
30         }
31     }
32 }
```

JDBC로 insert, select

요청 및 응답 흐름

- Controller는 "무엇을", DAO는 "어떻게 가져올지", JSP는 "어떻게 보여줄지" 담당



BookVo

```
1  package com.aitech.library.book;
2
3  public class BookVo {
4      private int b_no;
5      private String b_name;
6      private String b_author;
7      private String b_publisher;
8      private String b_publish_year;
9      private String b_isbn;
10     private String b_call_number;
11     private int b_rebookntal_able;
12     private String b_img;
13     private String b_reg_date;
14     private String b_mod_date;
15
16     //getter / setter 메뉴로 생성 Source -> Generate Getters and Setters...
17 }
```

BookDao - insert

```java
@Repository
public class BookDao {
    private DataSource dataSource;

    @Autowired
    public BookDao(DataSource dataSource) {
        this.dataSource = dataSource;
    }

    public void insertBook() {
        String sql = "INSERT INTO book(b_name, b_author, b_rental_able, b_reg_date, b_mod_date) " +
                     "VALUES(?, ?, ?, now(), now())";
        try {
            Connection conn = dataSource.getConnection();
            PreparedStatement pstmt = conn.prepareStatement(sql);
            pstmt.setString(1, "JDBC로 넣은 책");
            pstmt.setString(2, "홍길동");
            pstmt.setInt(3, 1);
            pstmt.executeUpdate();
            conn.close();
        } catch (SQLException e) {
            e.printStackTrace();
        }

    }
}
```

DBTestController 호출용 임시 매핑 URL 추가

```
38      @RequestMapping("/insertTest")
39      @ResponseBody
40      public String insertTest() {
41          bookDao.insertBook();
42          return "insert 완료";
43      }
```

BookDao - select

```
40    public List<BookVo> getAllBook() {
41        String sql = "SELECT * FROM book ORDER BY b_no";
42        List<BookVo> books = new ArrayList<>();
43
44        try {
45            Connection conn = dataSource.getConnection();
46            PreparedStatement pstmt = conn.prepareStatement(sql);
47            ResultSet rs = pstmt.executeQuery();
48            while (rs.next()) {
49                BookVo book = new BookVo();
50                book.setB_no(rs.getInt("b_no"));
51                book.setB_name(rs.getString("b_name"));
52                book.setB_author(rs.getString("b_author"));
53                book.setB_publisher(rs.getString("b_publisher"));
54                book.setB_publish_year(rs.getString("b_publish_year"));
55                book.setB_isbn(rs.getString("b_isbn"));
56                book.setB_call_number(rs.getString("b_call_number"));
57                book.setB_rental_able(rs.getInt("b_rental_able"));
58                book.setB_img(rs.getString("b_img"));
59                books.add(book);
60            }
61        } catch (SQLException e) {
62            e.printStackTrace();
63        }
64        return books;
65    }
```

BookController

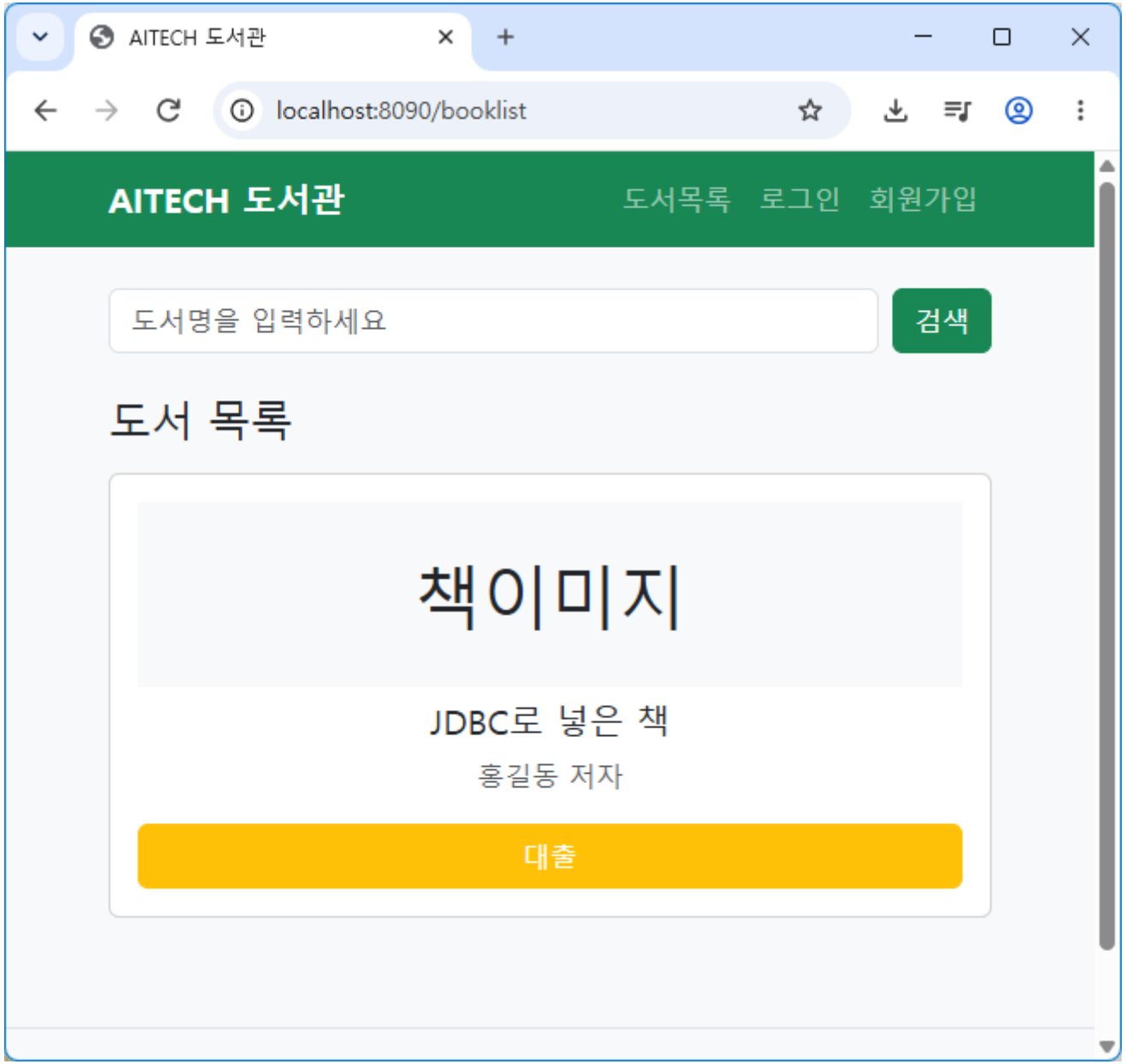
- HomeController의 RequestMapping의 booklist 삭제

```
10 @Controller
11 public class BookController {
12     private BookDao bookDao;
13 
14     @Autowired
15     public BookController(BookDao bookDao) {
16         this.bookDao = bookDao;
17     }
18 
19     @RequestMapping("/booklist")
20     public String bookList(Model model) {
21         List<BookVo> books = bookDao.getAllBook();
22         model.addAttribute("books", books);
23         return "book/book_list";
24     }
25 }
```

book_list.jsp

```jsp
<%@ page contentType="text/html; charset=UTF-8" pageEncoding="UTF-8"%>
<%@ taglib prefix="c" uri="jakarta.tags.core" %>
<%@ include file="../include/header.jsp"%>
<div class="d-flex gap-2 mb-4">
    <input type="text" class="form-control" placeholder="도서명을 입력하세요">
    <button class="btn btn-success flex-shrink-0">검색</button>
</div>
<h3 class="mb-3">도서 목록</h3>
<div class="row">
    <c:forEach var="book" items="${books}">
        <div class="col-md-4 mb-3">
            <div class="card">
                <div class="card-body text-center">
                    <div class="bg-light py-4 mb-2" style="font-size:40px">책이미지</div>
                    <h5>${book.b_name}</h5>
                    <p class="text-muted">${book.b_author} 저자</p>
                    <div class="d-grid">
                        <button class="btn btn-warning text-white">대출</button>
                    </div>
                </div>
            </div>
        </div>
    </c:forEach>
  <!-- 카드 2장 더 복사 (책만 다르게) -->
</div>
<%@ include file="../include/footer.jsp"%>
```

book_list.jsp



HomeController

권장 수정

```
@RequestMapping("/")
public String home(Model model) {
    return "redirect:/booklist";}
```

리턴값 두 종류 — 오늘 확실히 구분하기

`return "book/book_list"` → 뷰 이름. ViewResolver가 JSP를 표시

`return "redirect:/bookList"` → URL. 브라우저가 그 주소로 다시 요청을 보낸다 (재요청)

redirect는 목록 로직을 BookController 한 곳에만 두고 재사용하는 방법

이렇게도 되지만 비권장

```
@RequestMapping("/")
public String home(Model model) {
    List<BookVo> books = bookDao.getAllBook();
    model.addAttribute("books", books);
    return "book/book_list";
}
```

→ bookList()의 조회 로직 복사·중복. 검색이 추가되면 두 군데를 고쳐야 한다.

JDBC Template

JDBC Template로 축약

BookDao 기존 코드

전체 조회 getAllBook

```
public List<BookVo> getAllBook() {
    String sql = "SELECT * FROM book ORDER BY b_no";
    List<BookVo> books = new ArrayList<>();

    try {
        conn = dataSource.getConnection();    // ① 연결 열기
        pstmt = conn.prepareStatement(sql);
        rs = pstmt.executeQuery();
        while (rs.next()) {                   // ② 반복
            BookVo book = new BookVo();       // ③ 한 행 변환
            book.setB_no(rs.getInt("b_no"));
            book.setB_name(rs.getString("b_name"));
            book.setB_author(rs.getString("b_author"));
            /* ...컬럼 수만큼 계속... */
            books.add(book);
        }
    } catch (Exception e) {                   // ④ 예외 처리
        e.printStackTrace();
    }
    return books;    // 연결 닫기도 원래는 직접...
}
```

BookDao 축약 코드

BookDao — 같은 기능이 4줄로

```
public List<BookVo> getAllBook() {
    String sql = "SELECT * FROM book ORDER BY b_no";
    return jdbcTemplate.query(sql, new BookRowMapper());
}
```

왼쪽 코드의 변화

① 연결 열기·닫기 / ④ try-catch ➡ JdbcTemplate이 내부에서 대신 처리

② while(rs.next()) 반복 ➡ query()가 행 수만큼 알아서 반복

③ 한 행 → BookVo 변환 코드 ➡ **유일하게 살아남아 mapRow()로 이사**

"몇 줄이든 반복해 List로 모으는 일"은 JdbcTemplate의 몫,
"한 줄을 객체로 바꾸는 규칙"만 우리가 RowMapper에 정의.

BookRowMapper

```java
public class BookRowMapper implements RowMapper<BookVo> {
    @Override
    public BookVo mapRow(ResultSet rs, int rowNum) throws SQLException {
        BookVo book = new BookVo();
        book.setB_no(rs.getInt("b_no"));
        book.setB_name(rs.getString("b_name"));
        book.setB_author(rs.getString("b_author"));
        book.setB_publisher(rs.getString("b_publisher"));
        book.setB_publish_year(rs.getString("b_publish_year"));
        book.setB_isbn(rs.getString("b_isbn"));
        book.setB_call_number(rs.getString("b_call_number"));
        book.setB_rental_able(rs.getInt("b_rental_able"));
        book.setB_img(rs.getString("b_img"));

        return book;
    }
}
```

BookDao 수정

```java
@Repository
public class BookDao {
    private JdbcTemplate jdbcTemplate;

    @Autowired
    public BookDao(JdbcTemplate jdbcTemplate) {
        this.jdbcTemplate = jdbcTemplate;
    }

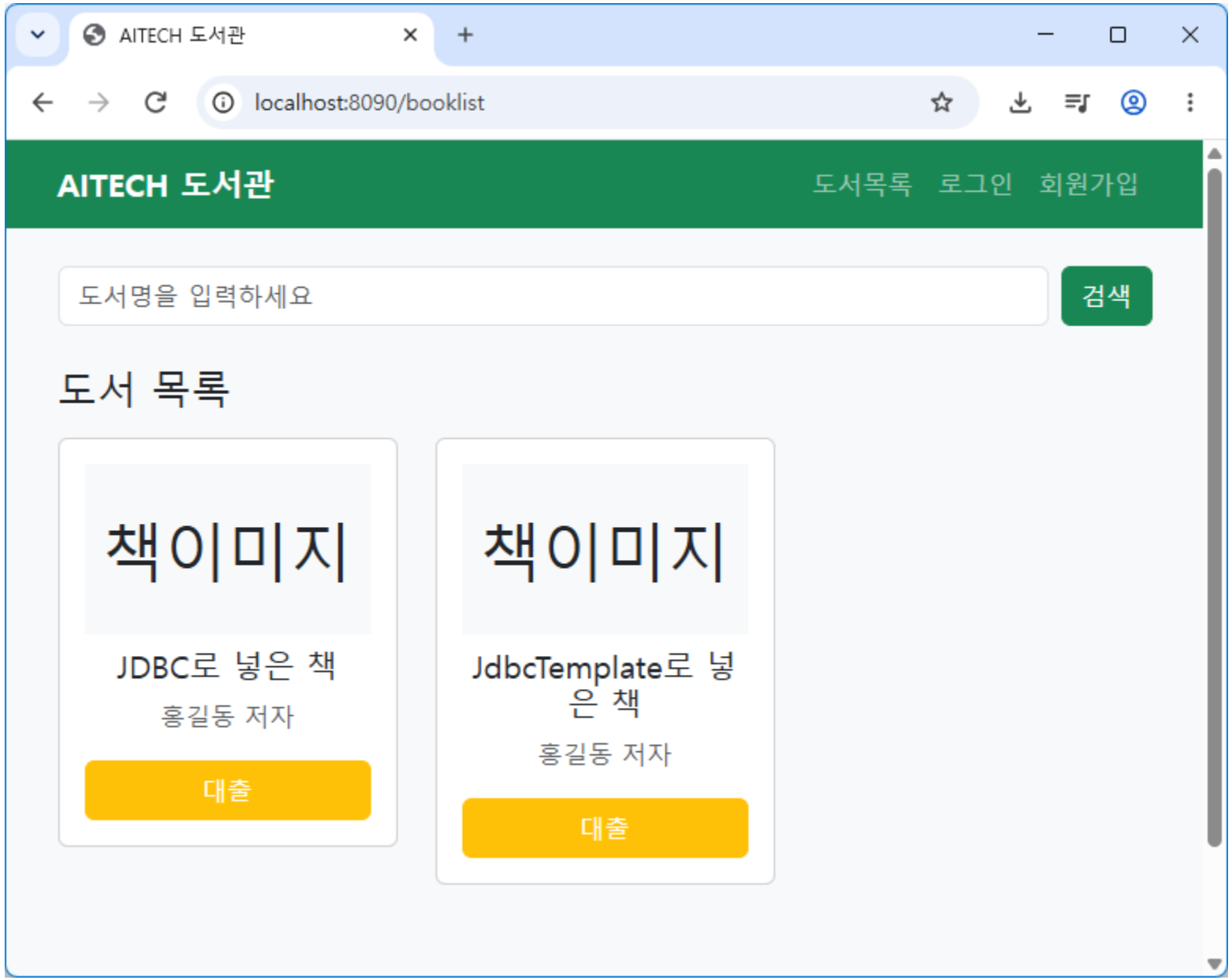
    public void insertBook() {
        String sql = "INSERT INTO book(b_name, b_author, b_rental_able, b_reg_date, b_mod_date) " +
                     "VALUES(?, ?, ?, now(), now())";
        jdbcTemplate.update(sql, "JdbcTemplate로 넣은 책", "홍길동", 1);
    }

    public List<BookVo> getAllBook() {
        String sql = "SELECT * FROM book ORDER BY b_no";
        return jdbcTemplate.query(sql, new BookRowMapper());
    }
}
```

DBconfig 추가

```
37      @Bean
38      public JdbcTemplate jdbcTemplate(DataSource dataSource) {
39          return new JdbcTemplate(dataSource);
40      }
41  }
```

book_list.jsp



register_book.jsp 수정

```
4     <div class="card shadow rounded-4">
5         <div class="card-body p-5">
6             <h3 class="card-title text-center mb-4">도서 등록</h3>
7 
8             <form method="post" action="registerbookconfirm"> <a%--추가--%>
9 
10                <%-- 기존 코드 그대로 --%>
11 
12                <div class="d-flex justify-content-center gap-3">
13                    <button type="submit" class="btn btn-success px-4">등록</button> <%-- 수정 --%>
14                    <a href="booklist" class="btn btn-secondary px-4">취소</a> <%-- 수정 --%>
15                </div>
16            </form> <%-- 추가 --%>
17        </div>
18    </div>
```

BookDao 수정

```java
@Repository
public class BookDao {
    private JdbcTemplate jdbcTemplate;

    @Autowired
    public BookDao(JdbcTemplate jdbcTemplate) {
        this.jdbcTemplate = jdbcTemplate;
    }

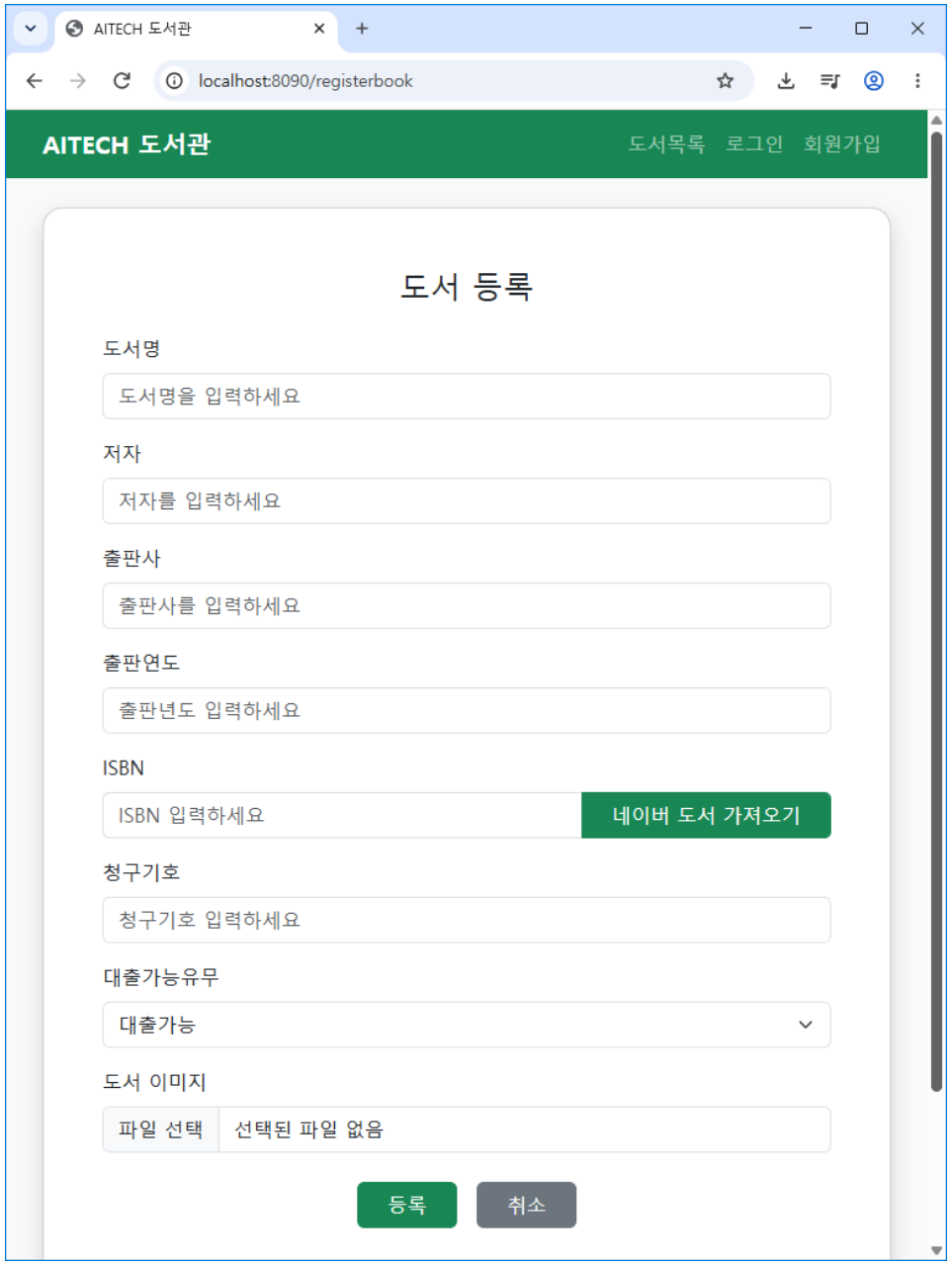
    public void insertBook(BookVo bookVo) {
        String sql = "INSERT INTO book(b_name, b_author, b_publisher, b_publish_year, b_isbn, b_call_number, b_rental_able, b_img, " +
                    "b_reg_date, b_mod_date) VALUES(?, ?, ?, ?, ?, ?, ?, ?, now(), now())";
        jdbcTemplate.update(sql, bookVo.getB_name(), bookVo.getB_author(), bookVo.getB_publisher(), bookVo.getB_publish_year(),
                bookVo.getB_isbn(), bookVo.getB_call_number(), bookVo.getB_rental_able(), bookVo.getB_img());
    }
```

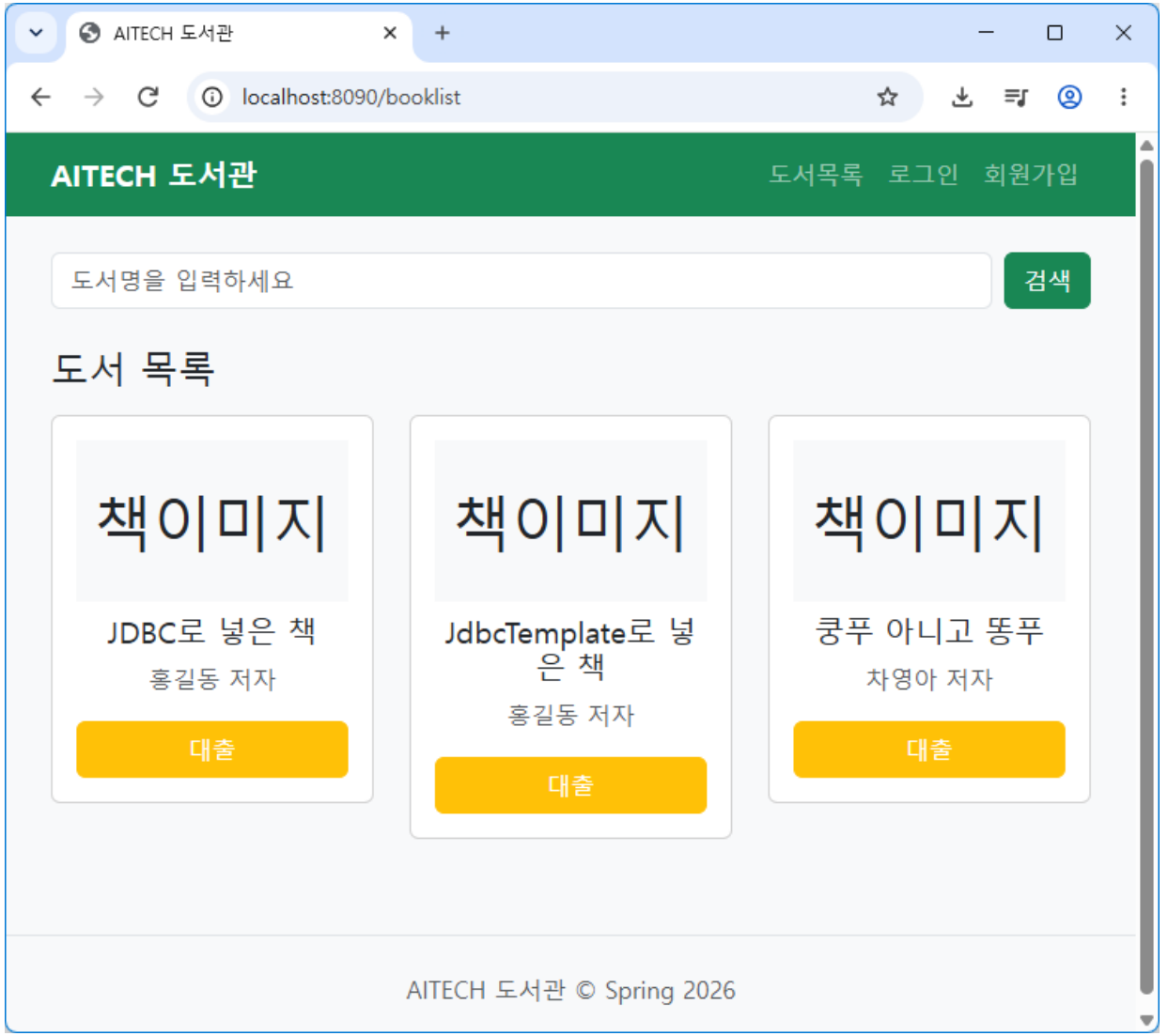
BookController 수정

- HomeController의 registerbook 삭제 후

```
26    @RequestMapping("/registerbook")
27    public String registerbook() {
28        return "admin/register_book";
29    }
30
31    @RequestMapping("/registerbookconfirm")
32    public String registerbookconfirm(BookVo bookVo) {
33        bookDao.insertBook(bookVo);
34        return "admin/register_book";
35    }
```

register_book.jsp / book_list.jsp





Thank you!